Constraint Satisfaction Problems
Solving CSP
Backtracking Search
Inference for CSP

Constraint Satisfaction Problem

AIMA (4th Global) Sections 5.1–5.3

Summary

- Introduction to CSP
- Backtracking
- Look-Ahead

Recap on finding solutions using search

◇ Assumptions: single agent, deterministic actions, fully observable state, (discrete state space)

Planning: sequences of actions

- We care about the path to reach a goal
- Paths have various costs
- Heuristics provide problem-specific guidance

Identification: assignments to variable

- the goal configuration is all we care about, not the path
- Local Search can be extremely useful (but no guarantees in general), can be used also in continuous spaces
- Constraint Satisfaction Problems (CSP) are a specialized class of identification problems

Constraint Satisfaction Problems

Constraint Satisfaction Problem

Constraint Satisfaction Problems

Solving CSP

Backtracking Search

Inference for CSP

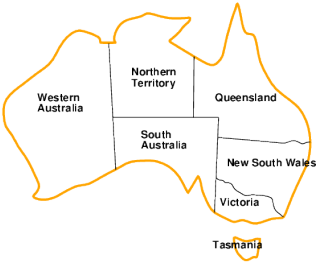
Standard Search Problem

- State is a black-box (can be any data structure)
- Goal test can be any function over states
- Successor function can be any function that maps a state to a set of successor states

Constraint Satisfaction Problems

- State is defined by a set of variables $X = X_i$ that can take values in a set of domains $D = D_i$
- Goal test is a set of constraints specifying valid combinations of values for subsets of variables
- We can use general-purpose solution strategies for CSP that exploit these features and are more efficient than standard search approaches

CSP Example: Map-Coloring



Variables WA, NT, Q, NSW, V, SA, T

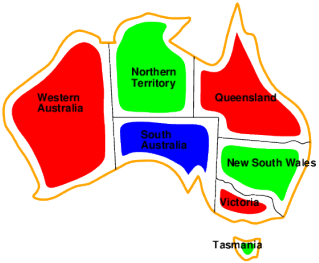
Domains $D_i = \{red, green, blue\}$

Constraints: adjacent regions must have different colors

e.g., $WA \neq NT$ (if the language allows this), or

$(WA, NT) \in \{(red, green), (red, blue), (green, red), (green, blue), \ldots\}$

Example: Map-Coloring contd.



Solutions are assignments satisfying all constraints, e.g.,
$\{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green\}$

Example: N-Queens

$\diamondsuit$ Place N queens on a NxN check board so that no couple of queens attack each other

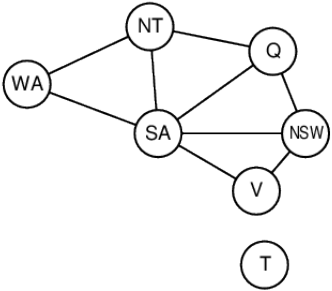
- $X : \{x_{i,j}\}$, $i \in \{1, \ldots, N\}$, $j \in \{1, \ldots, N\}$: one variable for each cell
- $D : \{D_{i,j}\}$ $D_i \in 0, 1$: queen is present or not
- $\forall\ i, j, k (X_{i,j}, X_{i,k}) \in \{(0,0), (0,1), (1,0)\}$: only one queen for each row
- $\forall\ i, j, k (X_{i,j}, X_{k,j}) \in \{(0,0), (0,1), (1,0)\}$: only one queen for each column
- $\forall\ i, j, k (X_{i,j}, X_{i+k,j+k}) \in \{(0,0), (0,1), (1,0)\}$: only one queen in diagonal (to the right)
- $\forall\ i, j, k (X_{i,j}, X_{i+k,j-k}) \in \{(0,0), (0,1), (1,0)\}$: only one queen in diagonal (to the left)
- $\sum_{i,j} X_{i,j} = N$

Example: N-Queens alternative formulation

- Consider only one key for each column
  - $X : \{x_i\}\ i \in \{1, \ldots, N\}$one variable for each queen in different columns
  - $D : \{D_i\}\ D_i \in \{1, \ldots, N\}$ row for queen in column i
  - $C : \{C_1, ..., C_k\}\ k \in \{1, \ldots, Nx(N-1)/2\}$ one constraint for each couple of variables, specifying allowed positions for the two queens
    - $C_1 : (x_1, x_2) \in \{(1,3), (1,4)(2,4), (3,1), (4,1), (4,2)\}$
    - $C_2 : (x_1, x_3) \in \{(1,2), (1,4), (2,1), (2,3), (2,4), (3,2), (3,4), (4,2), (4,3)\}$
    - $\ldots$
  - This formulation makes some constraints implicit
    - e.g., not possible to assign two queens to same column so no need to check

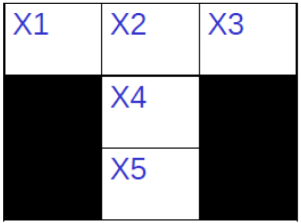
Constraint graph

◇ Primal graph: one node for each variable, one edge for each constraints that holds between the two variables

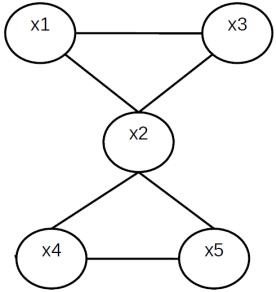


Example: Crossword

◇ Given crosswords configuration and possible words, assign letters to cells so to match the given configuration



Possible words: {MAP, ARC}
Only word of correct length

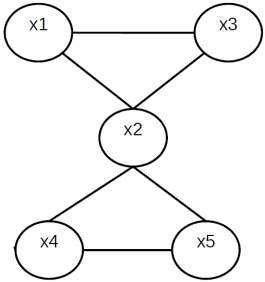


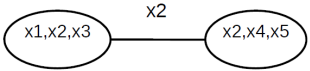
Di : letters of the alphabet
C1 [{x1,x2,x3},(MAP)(ARC)]
C2 [{x2,x4,x5},(MAP)(ARC)]

Non binary constraints

Example (Crossword puzzle: dual graph)

Di : letters of the alphabet
C1 [{x1,x2,x3},(MAP)(ARC)]
C2 [{x2,x4,x5},(MAP)(ARC)]



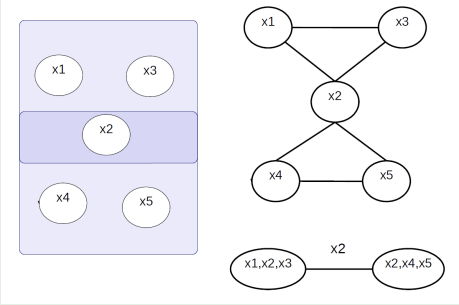


Hypergraphs and Dual Graphs

Relationships between hypergraphs and binary graphs

- Can always convert a hypergraph into a binary graph
- Each variable has an **exponentially larger domain**

Example (Crossword puzzle)

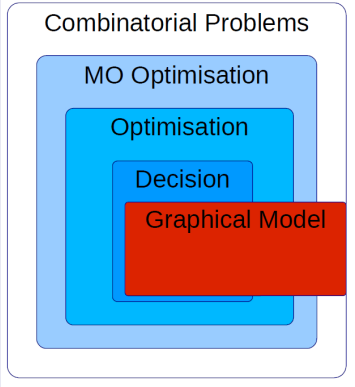


CSP a wider perspective

Combinatorial problems

Given a set of possible solutions **find the best one**

Space of solutions is huge, complete search is not possible



Graphical Models

Different types of combinatorial problems

- Decisions: colour a graph with k colours (Yes/No)
- Optimization: colour a graph with k colours minimizing conflicts
- MO-optimization: portfolio investment, minimize risk/maximize revenues
- Graphical Models
  - Set of variables, domains and **local** functions (constraints)
  - **Global** function is an aggregation of local functions
  - Solutions: assignment of variable that optimizes global function

Mathematical formulation of Constraint Networks

Definition of CN

A tuple of three elements $CN =< X, D, C >$

- $X = \{x_1, ..., x_n\}$: set of variables
- $D = \{D_1, ..., D_n\}$: set of domains
- $C = \{(S_1, R_1), ..., (S_m, R_m)\}$ set of constraints
- each constraint (S_i, R_i)
  - $S_i \subseteq X$ **scope**: variables involved in R_i
  - R_i subset of cartesian product of variables in S_i
  - R_i expresses allowed tuples over S_i
- **Solution**: assignment of **all** variables that satisfies **all** constraints
- Tasks: consistency check, find one or all solutions, count solutions, find best solution (optimization)

Solutions and partial solutions

Partial Solution: assignment of a subset of variables

- Consistent partial solution: partial sollution that satisfies all the constraint whose scope contains no unassigned variables
- **A consistent partial solution is not necessarily part of a solution**

Example (4-Queen example)

- $X : \{x_1, ..., x_4\}$ one variable for each queen in different columns
- $D : \{D_1, ..., D_4\}$ $D_i \in [1, ..., 4]$ row for queen i
- $C : \{C_1, ..., C_6\}$ one constraint for each couple of variables, allowed positions for the two queens
- partial consistent solution $\{x_1 = 1, x_2 = 4, x_3 = 2\}$, solution $\{x_1 = 2, x_2 = 4, x_3 = 1, x_4 = 3\}$

Standard search formulation for CP

Naive choice

States are defined by the values assigned so far

◇ Initial state: the empty assignment, $\{\}$

◇ Successor function: assign a value to an unassigned variable that does not conflict with current assignment.

$\Longrightarrow$ fail if no legal assignments (not fixable!)

◇ Goal test: the current assignment is complete

1. This is the same for all CSPs! 😊
2. Every solution appears at depth n with n variables
   $\Longrightarrow$ use depth-first search
3. Path is irrelevant, so can also use complete-state formulation
4. $b = (n - \ell)d$ at depth ℓ, hence $n!d^n$ leaves!!!! ☹

Backtracking search

A better formulation

Variable assignments are commutative, i.e.,
[$WA = red$ then $NT = green$] same as [$NT = green$ then $WA = red$]
Only need to consider assignments to a single variable at each node
$\Longrightarrow$ $b = d$ and there are d^n leaves

◇ Depth-first search for CSPs with single-variable assignments is called backtracking search

◇ Backtracking search is the basic uninformed algorithm for CSPs. Can solve n-queens for $n \approx 25$

◇ Variable ordering counts for performance

Backtracking search

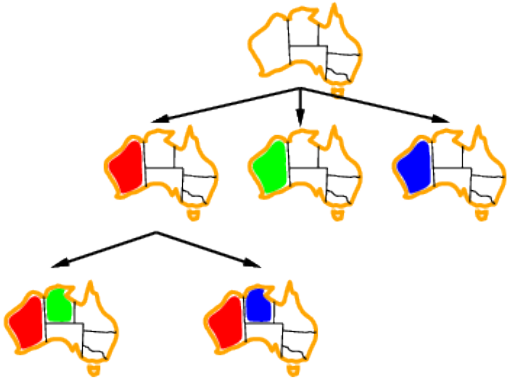
```
function Backtracking-Search(csp) returns solution or failure
    return Backtrack({ }, csp)


function Backtrack(assignment, csp) returns solution or failure
    if assignment is complete then return assignment
    var ← Select-Unassigned-Variable(csp)
    for each value in Order-Domain-Values(var, assignment, csp) do
        if value is consistent with assignment then
            add {var = value} to assignment
            inferences ← Inferences(csp, var, value)
            if inferences ≠ failure then
                add inferences to assignment
                result ← Backtrack(assignment, csp)
                if result ≠ failure then
                    return result
                endif
            endif
        endif
        remove {var = value} and inferences from assignment
    endfor
return failure
```

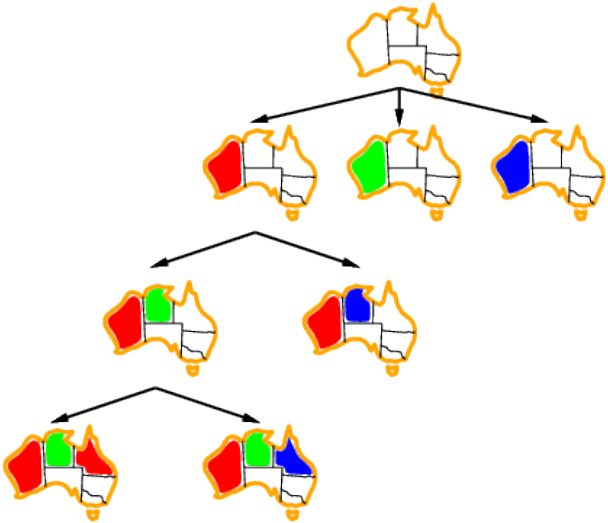
Backtracking example

Backtracking example

Backtracking example



Backtracking example



Improving Backtracking

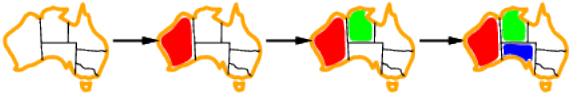
◇ General Goal: Reducing size of explored search space

Ingredients

- ordering variables and variables' values
- local consistency (e.g., arc or path consistency)
- look-ahead, predict future inconsistencies
- (look-back, where to backtrack)
- tree decomposition, exploit problem structure

Ordering Variables: Minimum remaining values

Minimum remaining values (MRV):
choose the variable with the fewest legal values

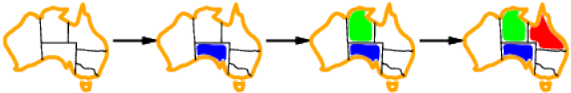


Ordering Variables: Degree heuristic

Tie-breaker among MRV variables

Degree heuristic:

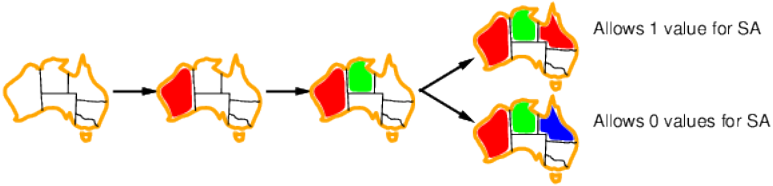
choose the variable with the most constraints on remaining variables



MRV and DH can be applied to any CSP

Value ordering: Least constraining value

Given a variable, choose the least constraining value:
the one that rules out the fewest values in the remaining variables



Combining these heuristics makes 1000 queens feasible

Gains depend on the specific problem

Inference: make hidden constraints explicit

- We can reason about constraints and infer new constraints
- Example: Graph colouring network $\mathcal{R}$
  - Given $\{x_1, x_2, x_3\}$, $\{D_1 = D_2 = D_3\}$, $D_i = \{R, B\}$
    $C = \{C_1 : (x_1 \neq x_2), C_2 : (x_2 \neq x_3)\}$,
  - we can infer $C_3 : (x_1 = x_3)$
  - we do not need to consider any assignment where $(x_1 \neq x_3)$

Trade-off between inference and search

- Tighter constraints $\rightarrow$ smaller search space (more constraints help in the search process!)
- Adding constraints requires more computation
- Each time a new variable is assigned need to check many more constraints
- If only binary constraints we never have more than $O(n)$ checks
- If r-ary constraints then we could have $O(n^{r-1})$ checks

Backtrack Free Search

Backtrack Free Network

- A network $\mathcal{R}$ is backtrack free if every leaf is a goal state
- A DFS on a backtrack free network ensure a complete consistent assignment
- E.g. $\mathcal{R} + C_3 \rightarrow$ backtrack free network

Look-Ahead

Look-Ahead Schemes

- Given approximate inference (arc consistency, path-consistency)
- Foresee impact of next move (which variable, which value)
- Impact: how next move restricts future assignments
- Efficient way to update information for choosing next variable/value
  - Can efficiently compute remaining legal values given current assignment

Look-Ahead Strategies

Strategies

- Forward Checking
  - check unassigned variables separately
- Arc consistency look-ahead
  - propagate arc consistency

Look-ahead: Discussion

Discussion

- Incur extra cost for assigning values
  - need to propagate constraints
- Can restrict search space significantly
  - e.g., discover that a value makes a sub-problem inconsistent
  - remove values from future variables' domains
- Usually no changes on worst case performance: trade-off between costs and benefits

Forward Checking

Forward Checking

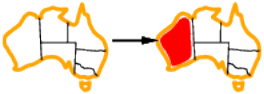
- most limited form of constraint propagation
- propagates the effect of a selected value to future variables separately
- if domains of one of future variables becomes empty, try next value for current variable.

Forward checking

Idea: Keep track of remaining legal values for unassigned variables
Terminate search when any variable has no legal values

Forward checking

Idea: Keep track of remaining legal values for unassigned variables
Terminate search when any variable has no legal values

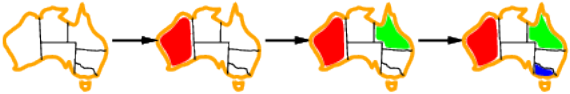


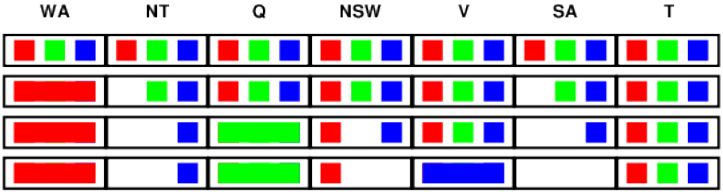
Forward checking

Idea: Keep track of remaining legal values for unassigned variables
Terminate search when any variable has no legal values

Forward checking

Idea: Keep track of remaining legal values for unassigned variables
Terminate search when any variable has no legal values





Forward Checking Example

Example (Graph Colouring Example R_{GC})

- Variables: $x_1, x_2, x_3, x_4, x_5, x_6, x_7$,
- Domains: $D_{x_1} = \{R, B, G\}, D_{x_2} = D_{x_5} = \{B, G\}, D_{x_3} = D_{x_4} = D_{x_7} = \{R, B\}, D_{x_6} = \{R, G, Y\}$
- Constraints: $x_1! = x_2, x_1! = x_3, x_1! = x_4, x_1! = x_7, x_2! = x_6, x_3! = x_7, x_4! = x_5, x_4! = x_7, x_5! = x_6, x_5! = x_7$
- $x_1 = red$ reduces domains of x_3, x_4, x_7
- $x_2 = blue$ no effects
- $x_3 = blue$ (only available) makes x_7 empty $\rightarrow$ x_3 dead-end

Complexity of Forward Checking

Complexity of Select Value Forward Checking

- $O(ek^2)$
- e_u consistency check for each value of each future variable x_u
- k value for each future variables $O(e_u k)$
- $\sum_u e_u = e$ then $O(ek)$
- k value for the current variable

Arc Consistency Look-Ahead

Arc Consistency Look ahead

- force full arc consistency on all remaining variables
- select a value for current variable $x_i = a$
- force arc constraint on all unassigned variables with $x_i = a$
- An arc $x_k \rightarrow x_j$ is consistent iff for every assignment of x_k there is at least one assignment of x_j that is consistent with the (x_k, x_j) constraint.
- Forcing arc consistency: if no value of x_j is consistent with a given value of $x_k = c$ we remove c from the domain of x_k
- Forward checking: enforcing consistency from every unassigned variable to a new assignment
- Note: in forward checking we never check consistency between non assigned variables, in Arc consistency look-ahead for every new assignment we check arc consistency between each couple of variables.

The power of AC

◇ Sudoku as a Constraint Network:
Variables: 81 cells ($\{A1, A2, \ldots, I9\}$), each cell has domain $D = \{1, \ldots, 9\}$
Constraints: AllDiff(...) for
every rows ($AllDiff(A1, A2, \ldots, A9)$)
columns ($AllDiff(A1, B1, \ldots, I1)$)
boxes
($AllDiff(A1, A2, A3, B1, B2, B3, C1, C2, C3)$)

◇ In this case we can solve the entire puzzle **just by forcing AC**

◇ Focus on column 6 as an example

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

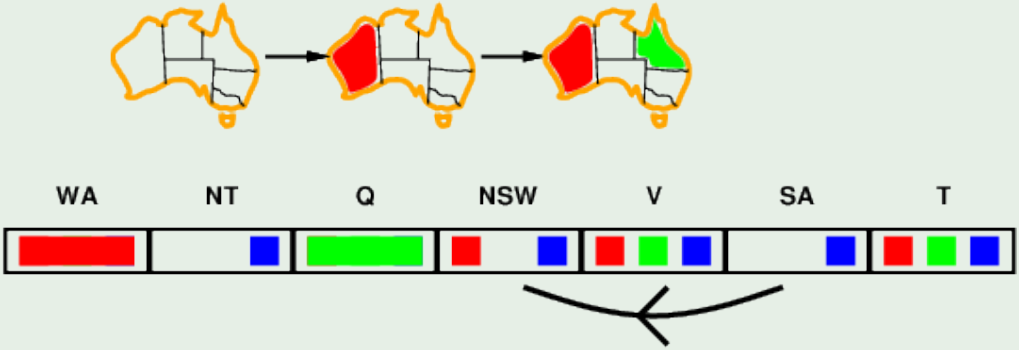
Arc Consistency Look-Ahead Complexity

Arc Consistency Look Ahead

- Best algorithm for AC is $AC-4$ complexity $O(ek^2)$
  - AC-1, AC-3 and AC-4 are systematic algorithms to efficiently check constraints and make sure the constraint network is AC (i.e., every constraint is arc consistent).
- worst case complexity for Arc Consistency look-ahead is $O(ek^3)$
  - need to run AC-4 for every possible assignment of the current variable

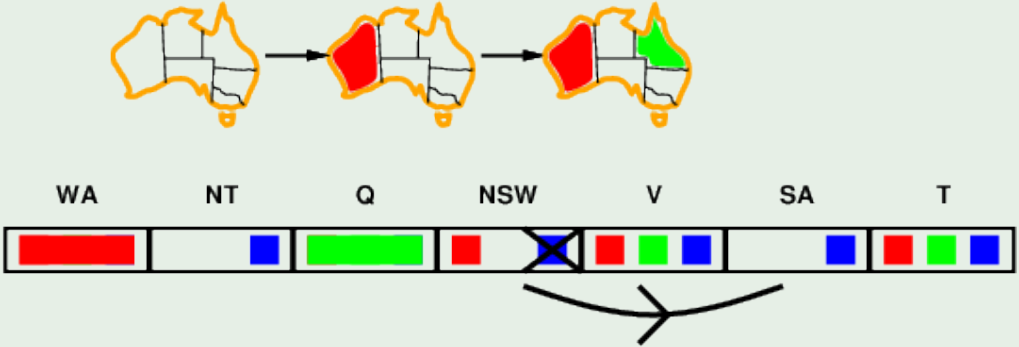
Example of AC Look-Ahead

Example (AC Look-Ahead for Map Colouring)



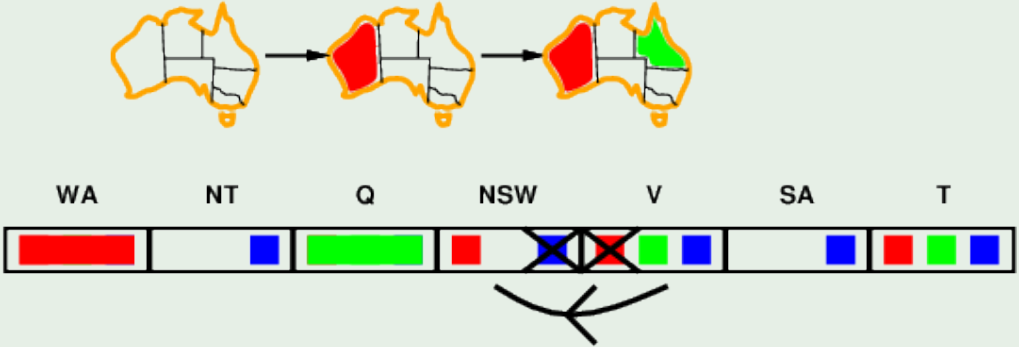
Example of AC Look-Ahead

Example (AC Look-Ahead for Map Colouring)



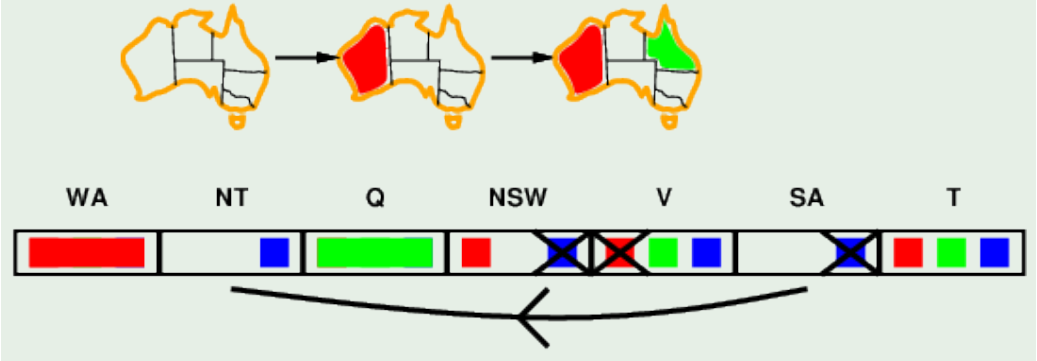
Example of AC Look-Ahead

Example (AC Look-Ahead for Map Colouring)



Example of AC Look-Ahead

Example (AC Look-Ahead for Map Colouring)



Enforcing Arc Consistency for Networks

Definition (Arc Consistent Network)

- R_{x_i,x_j} is arc consistent iff x_i arc consistent w.r.t. x_j and x_j arc consistent w.r.t. x_i
- $\mathcal{R}$ is arc consitent iff all its constraints are arc consitent
- Can easily ensure that a couple of variables are arc consistent (Revise procedure)
- But revising arc consistency on a variable might make another variable not-arc consistent
- Need systematic procedures to ensure Arc Consistency on networks.

Insights on Arc Consistency

Empty Domain and Arc Consistency

- Arc consistency + empty domain $\rightarrow$ inconsistent problem
- Arc consistent + all domains are not empty $\not\rightarrow$ consistent problem
- Arc consistency is not complete
  - It checks only single (binary) constraints and single domain constraint
- Arc consisteny + all domains are not empty + **No Cycles for primal graphs** + **Binary Constraints** $\rightarrow$ consistent backtrack free network

Exploiting problem structure in Look ahead

Definition (Cycle Cutset)

Given an undirected graph, a subset of nodes in the graph is a cycle cutset iff its removal result in an acyclic graph

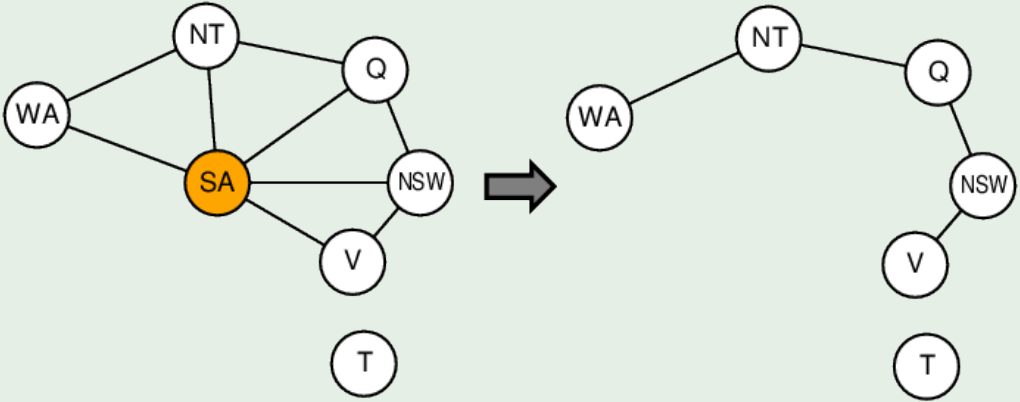
Exploiting problem structure

- Once a variable is assigned it can be removed from the graph (conditioning)
- If we remove a cycle-cutset the rest of the problem is a tree
- Can use arc consistency to solve that sub-problem
- We need to check all possible assignments of cycle-cutset variables and do arc propagation
- Complexity is still exponential but in the size of the cycle-cutset!

Exploiting problem structure: example

Example (Cycle Cut Set for Australia map colouring)

Conditioning: instantiate a variable, prune its neighbors' domains



Cutset conditioning: instantiate (in all ways) a set of variables

Exercise: AC Look-Ahead

AC for Graph Colouring Example R_{GC}

- Variables: $x_1, x_2, x_3, x_4, x_5, x_6, x_7,$
- Domains: $D_{x_1} = \{R, B, G\}, D_{x_2} = D_{x_5} = \{B, G\}, D_{x_3} = D_{x_4} = D_{x_7} = \{R, B\}, D_{x_6} = \{R, G, Y\}$
- Constraints: $x_1! = x_2, x_1! = x_3, x_1! = x_4, x_1! = x_7, x_2! = x_6, x_3! = x_7, x_4! = x_5, x_4! = x_7, x_5! = x_6, x_5! = x_7$
- show that by using AC look ahead we would stop immediately after the assignment $x_1 = R$

Exercise: Cycle cut set

Cycle cut set for Graph Colouring Example R_{GC}

- Variables: $x_1, x_2, x_3, x_4, x_5, x_6, x_7,$
- Domains: $D_{x_1} = \{R, B, G\}, D_{x_2} = D_{x_5} = \{B, G\}, D_{x_3} = D_{x_4} = D_{x_7} = \{R, B\}, D_{x_6} = \{R, G, Y\}$
- Constraints: $x_1! = x_2, x_1! = x_3, x_1! = x_4, x_1! = x_7, x_2! = x_6, x_3! = x_7, x_4! = x_5, x_4! = x_7, x_5! = x_6, x_5! = x_7$
- Use the Cycle cut set algorithm to solve this graph colouring problem.